SOFTWARE DEVELOPMENT PROJECT

Final Report

AgroAdvisor: an AI-assisted farm advisory platform that helps smallholder farmers prepare for severe weather

Prepared by:

[Student name]

[Student number]

Department of Computer Science & Information Technology

Sol Plaatje University

Submitted to:

Dr Silas Verkijika

Lecturer: Project (NPRJ740)

Department of Computer Science & Information Technology

Sol Plaatje University

24-26 Scanlan St, New Park, Kimberley, 8301

[Submission date]

Yellow-highlighted text marks details only the student can supply or evidence that has not been collected yet.

TABLE OF CONTENTS

EXECUTIVE SUMMARY 4
1. PROJECT CONTEXT 4
2. PROBLEM, PURPOSE AND SUCCESS CRITERIA 4
2.1 Problem Statement 4
2.2 Project Purpose 5
2.3 Success Criteria 5
3. SOLUTION OVERVIEW AND SCOPE 6
3.1 Solution Overview 6
3.2 Final Release Scope 6
3.3 Excluded and Deferred Capabilities 7
3.4 Known Limitations and Open Issues 7
4. SYSTEM REQUIREMENTS SPECIFICATION 7
4.1 Functional Requirements 7
4.2 Non-Functional Requirements 8
4.3 Business Rules 9
4.4 Acceptance Criteria for Included Requirements 9
5. SYSTEM DESIGN SPECIFICATION 10
5.1 Use Cases and Workflows 10
5.2 Architecture and Technology Stack 14
5.3 Key Design Decisions 15
5.4 Application Class Design 16
5.5 User Interface and Interaction 17
5.6 Data Design 20
5.7 Security, Privacy and Access Control 22
5.8 Deployment and Runtime Design 22
6. SYSTEM IMPLEMENTATION 23
6.1 Codebase and Repository Structure 23
6.2 Implementation of Core Capabilities 23
6.3 APIs and Integrations 24
6.4 AI/ML Component 24
7. VERIFICATION AND TESTING 24
7.1 Test Approach 24
7.2 Functional and Business-Rule Verification 25

7.3 Non-Functional and Security Verification 25
7.4 Defects and Known Issues 26
8. SYSTEM SETUP AND OPERATION 26
8.1 Prerequisites and installation 26
8.2 Environment configuration 26
8.3 Run commands 26
8.4 Seeding and demo accounts 27
9. SOLUTION EVALUATION 27
10. REFERENCES 27
11. APPENDICES 28
11.1 Appendix A: Source code repository 28
11.2 Appendix B: Deployed system 28
11.3 Appendix C: Supporting material 28

EXECUTIVE SUMMARY

Most food in South Africa is not grown by large commercial farms alone. Statistics South Africa counted 2,463,429 agricultural households in the 2022 census, and many of them farm on a small scale with little access to personal, timely advice (Statistics South Africa, 2022). Weather is the biggest risk they cannot control. In September 2026 the South African Weather Service and the Department of Agriculture both warned that a developing El Niño could bring heat, dry spells and storms in the 2026/27 season (South African Weather Service, 2026; Department of Agriculture, 2026).

AgroAdvisor is a web application for these farmers. A farmer saves a simple farm profile (location, size, crops and livestock). The system then reads the seven-day forecast for that exact place and turns it into plain advice and severe-weather warnings, for example what to do about heat at 35 °C or more, or frost at 2 °C or less, and when. It does not only display the weather. It also offers short reviewed video lessons, a private farm memory, and a community map where nearby farmers share short local notes.

The final system has a React and TypeScript client, a Hono API on a Cloudflare Worker, and Firebase Authentication and Cloud Firestore protected by Security Rules. Weather comes from Open-Meteo. Google Gemini identifies a crop or animal from a photo, and Azure Speech reads warnings aloud. The advice and risk engine is rule-based, using thresholds kept in a knowledge-base file.

Outcome: 16 of the 17 planned functional requirements are built, one (background push notifications) is not, and Microsoft sign-in is not enabled. The code builds and type-checks cleanly, 6 Firestore security-rule tests and 15 unit tests pass, and a live validation test behaved correctly. Performance has not been measured and the system has been assessed running locally, not on a public URL. These limits are listed in Known Limitations and Open Issues and the Solution Evaluation.

1. PROJECT CONTEXT

- Domain: agricultural advisory and climate-risk communication for smallholder and small-scale farmers in South Africa. The system combines weather forecasting data, simple agronomic rules and farmer-to-farmer knowledge sharing.
- Users: farmers only. There is one type of user. There are no administrator, buyer or other roles. Farmers use the system mostly on a phone, so the interface is mobile-first.
- Basis of the project: a general application domain. It is not built for one named organisation. The sample farmers in the database (demo-farmer-01 to demo-farmer-15) are invented for testing and are not real people.

2. PROBLEM, PURPOSE AND SUCCESS CRITERIA

2.1 Problem Statement

Small-scale farmers farm close to the weather. When a heatwave, frost, heavy rain or storm arrives without a warning they can act on, they lose crops and animals, and a household that grows part of its own food loses that food and the money spent on seed and feed. South Africa has about 2.46 million agricultural households (Statistics South Africa, 2022), but only about 40,000 commercial farming units (Statistics South Africa, 2017), so most farming households are small and do not have a private adviser.

The gap is advice, not data. Weather forecasts are widely available, but a number such as "37 °C" does not tell a farmer what to do for tomatoes or for chickens, or when. Public extension support is stretched: a study of the Eastern Cape found that 80.2% of the extension practitioners surveyed had no training in climate-change adaptation, and that farmers preferred personal channels (60.3%) over printed or broadcast ones (Makamane, 2023).

The risk is now rising. The World Meteorological Organization expects a strong El Niño in 2026/27 (World Meteorological Organization, 2026), and the 2015/16 El Niño cut South African maize prospects sharply (USDA Foreign Agricultural Service, 2016). Not every El Niño causes drought, but the warning is a reason to prepare early. Food inflation is currently low (Farmer's Weekly, 2026), so the argument for the system is resilience, not a price crisis: households that can grow part of their own food and protect it from the weather depend less on outside help.

A software solution is justified because the work is repetitive and local: the same forecast must be re-read every day against each farm's crops and animals. A program can do this for every farmer at no extra cost, in plain language, on the phone they already own.

Evidence note: the statements above come from the published sources in the References. The statement that many new small-scale farmers do not yet know how to farm well is a working hypothesis. [Add the findings of the farmer interviews here, or remove this sentence if no interviews were held.]

2.2 Project Purpose

The purpose is to help small-scale farmers protect their crops and animals from severe weather and to farm better, by turning a farm-specific seven-day forecast into clear, actionable advice and warnings, and by supporting them with short lessons, a private record of what worked, and local information from nearby farmers.

2.3 Success Criteria

ID	Success criterion	How it is verified
SC-1	A farmer can register, sign in, save a farm profile and see advice for that farm on the dashboard.	Live use with a real account; unit tests for validation (Verification and Testing)
SC-2	Severe weather (heat, frost, heavy rain, strong wind, storm) produces a warning that says what to do, when and why, using the documented thresholds.	Unit tests TC-U01 to TC-U09; warning pop-up screenshot
SC-3	Each farmer can read and change only their own data, and community notes cannot be forged or reported twice.	Six Firestore rule tests; unauthenticated API checks
SC-4	Invalid input is refused with a clear message and nothing wrong is saved.	Unit tests TC-U10 to TC-U15; live validation test
SC-5	The project builds and type-checks cleanly and can be set up from the written instructions.	Client build, client and worker type checks (Verification and Testing)

3. SOLUTION OVERVIEW AND SCOPE

3.1 Solution Overview

AgroAdvisor is a mobile-first web application. A farmer signs in, saves a farm profile and lands on a dashboard that shows the farm name, the current weather, a seven-day outlook with a risk note for each day, and a warning icon whose colour shows the most serious expected event. Selecting the warning opens a short explanation of what to do, when and why, which can be read aloud. Other pages hold the farm profile, the Learn videos, the farm memory and the community map.

Figure 1 shows the system context: the farmer is the only user, and the system exchanges data with Firebase, Open-Meteo, Google Gemini, Azure Speech, YouTube and OpenStreetMap.

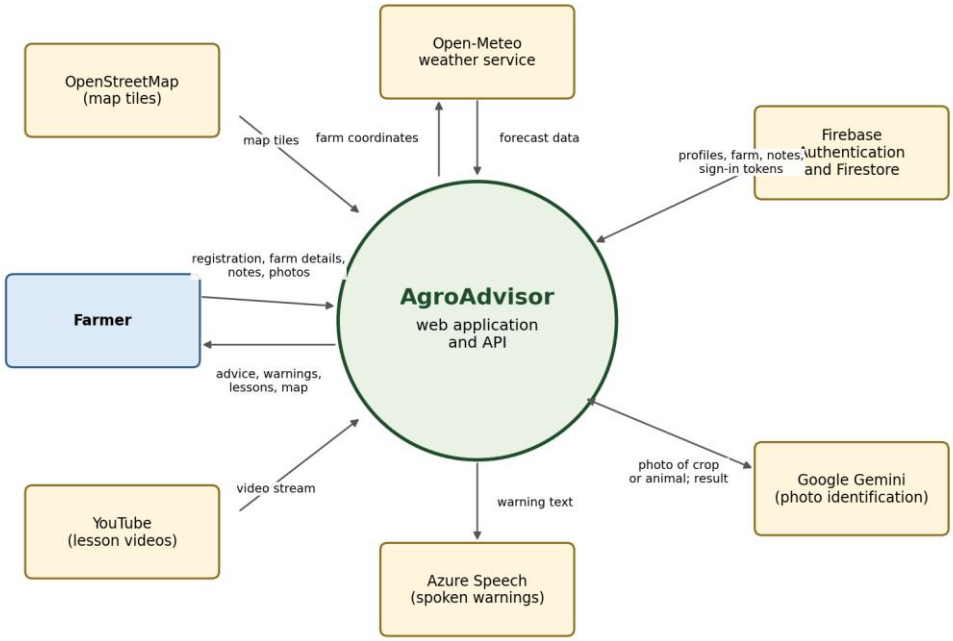


Figure 1: System context diagram

3.2 Final Release Scope

Included capability	What the farmer can do
Accounts	Register and sign in with email and password, or with Google.
Farm profile	Save one farm: name, nearest town, map location, size, crops, livestock and an optional photo. Identify a crop or animal from a photo. Estimate plot size by walking the boundary.
Weather advice	See current weather and a seven-day outlook interpreted for the farm.
Severe-weather warnings	See a coloured warning, a pop-up when the event is today or tomorrow, and hear it read aloud.
Learn	Watch reviewed video lessons by topic and mark them complete.
Farm memory	Record weather events, planting and what worked, with a weekly summary.
Community map	Add and see short local notes within 25 km, report another farmer's note, and remove your own.
Languages	Switch the interface between English, Afrikaans, isiZulu, Sesotho and Setswana.

3.3 Excluded and Deferred Capabilities

Capability	Status	Reason
Background push notifications when the app is closed (FR-17)	Deferred	Needs a Firebase service-account key held as a Worker secret. The code path for service-account tokens exists but was not exercised.
Microsoft sign-in	Deferred	The button exists, but the provider is not enabled because it needs an Azure app registration.
Month-by-month and seasonal advice	Deferred	The forecast source gives a seven-day horizon. Seasonal advice would need a seasonal-outlook data source and is planned, not built.
Crop suggestions for a farm	Deferred	Not yet approved as a feature.
Buyer, administrator or other roles	Excluded	The project serves farmers only.
Selling produce, payments, marketplace	Excluded	Outside the problem being solved.

3.4 Known Limitations and Open Issues

ID	Limitation or open issue	Effect
LIM-1	The assessed system runs locally. No public URL has been deployed or tested.	Hosting behaviour on Cloudflare Pages and Workers is untested.
LIM-2	No load-time or performance measurements were taken (NFR-07).	Performance claims are design targets only.
LIM-3	Authenticated API calls were not tested with a scripted client. Browser testing from another origin was blocked by CORS, and the test tool hides the ID token.	Validation and error responses of the vision and speech routes are known from the code, not from measured calls.
LIM-4	The wrong-password and login-error screens were not exercised in a live test during final testing.	Only the validation unit tests cover them.
LIM-5	The client npm audit reports 5 high-severity findings that are not yet fixed.	A dependency update is needed before public release.
LIM-6	Not every Learn video has been fully watched by the developer. [Confirm and record that each video was reviewed.]	Lessons are shown only when marked reviewed and published.
LIM-7	Sample data (15 demo farmers and their community notes) is present in the database.	It must be deleted before real use. All IDs start with demo-.
LIM-8	Advice is rule-based and covers only the crops and animals in the knowledge base. Photo identification can be wrong.	Farmers can always type a name instead.

4. SYSTEM REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

Priority uses MoSCoW (M = Must, S = Should, C = Could). Release status is the state in the final release.

ID	Requirement	Priority	Release status
FR-01	A farmer can register with full name, email and password and sign in with email and password.	M	Included
FR-02	A farmer can sign in with a Google account.	S	Included (Microsoft deferred)

ID	Requirement	Priority	Release status
FR-03	A farmer can create and edit one farm profile: name, nearest town, map location, size, crops, livestock with counts and an optional photo.	M	Included
FR-04	A farmer can set the farm location by tapping the map or using the device location.	M	Included
FR-05	A farmer can estimate plot size by walking the boundary. GPS points are not saved.	C	Included
FR-06	A farmer can add a crop or animal by photo identification or by typing its name.	S	Included
FR-07	The dashboard shows current weather next to the farm name.	M	Included
FR-08	The system shows a seven-day outlook with an interpretation and risk note for each day.	M	Included
FR-09	The system shows a severe-weather warning with what to do, when and why, and can read it aloud.	M	Included
FR-10	The farm map shades nearby estimated temperatures.	C	Included
FR-11	A farmer can browse reviewed and published video lessons by topic.	M	Included
FR-12	A farmer can mark a lesson complete and it is saved in the farm memory.	S	Included
FR-13	A farmer can record and delete weather, planting and success entries and see a weekly summary.	S	Included
FR-14	A farmer can add a local observation within 25 km of the farm and see visible nearby notes.	S	Included
FR-15	A farmer can report another farmer's note; a note is hidden after two different reports; the author can remove their own.	S	Included
FR-16	A farmer can change the interface language.	S	Included
FR-17	The system sends a background push notification when severe weather is forecast.	C	Deferred

4.2 Non-Functional Requirements

ID	Quality	Requirement	Verification
NFR-01	Reliability	If the weather service or Worker is unavailable the dashboard shows a clear error with a retry and no wrong data. A failed forecast never changes saved data.	Code inspection; not tested live
NFR-02	Availability	Uses managed services (Firebase, Cloudflare, Open-Meteo), so no server is maintained by the farmer.	Design review
NFR-03	Security	A farmer can read and write only their own data (Security Rules). The Worker verifies the Firebase ID token on every request. Secrets are never in source code.	Rule tests; API checks (Verification and Testing)
NFR-04	Security	Community notes carry no names. Rules limit note length, 25 km radius, one report per farmer per note and the report count.	Six rule tests
NFR-05	Maintainability	Strict TypeScript, no 'any', Firestore access only through service modules, thresholds in one knowledge-base file, one shared API response helper.	Type checks
NFR-06	Portability	Runs in any modern browser on phone or computer, with no app-store install.	Browser use
NFR-07	Performance	Mobile-first, split build, map loaded only when needed. [No measured load time yet.]	Not yet measured

ID	Quality	Requirement	Verification
NFR-08	Usability	Plain-language advice, 44 px touch targets, labelled controls, loading, empty and error states.	Screenshots; manual review
NFR-09	Cost	Free tiers only: Firebase Spark, Cloudflare free, Open-Meteo. No Cloud Functions.	Design review

4.3 Business Rules

ID	Business rule	Enforced by
BR-01	Each farmer has exactly one farm profile, stored under their own ID.	Firestore rules (farms/{ownerId})
BR-02	A farm size must be greater than 0 and at most 10,000 hectares, and a farm needs a name.	Client validation and Firestore rules
BR-03	Heat warning at 35 °C or more; frost warning at 2 °C or less; heavy rain at 50 mm, or 30 mm with 70% probability; strong wind at gusts of 60 km/h or more; storm for weather codes 95, 96 and 99.	Knowledge base (weather-risks.ts)
BR-04	A warning pop-up is shown only when the event is today or tomorrow.	Advice engine
BR-05	Only lessons marked both reviewed and published are visible. Farmers cannot write lessons.	Firestore rules
BR-06	A community note must be within 25 km of the author's farm and no longer than 240 characters.	Firestore rules
BR-07	A farmer cannot report their own note, and can report any note only once.	Firestore rules
BR-08	A note becomes hidden when two different farmers have reported it. Only the author can delete it.	Firestore rules and transaction
BR-09	A photo is accepted only as JPEG, PNG or WebP of at most 6 MB. A result with confidence 0.8 or higher is 'identified', 0.5 to 0.79 is 'uncertain', and lower is 'unknown'.	Worker vision route

4.4 Acceptance Criteria for Included Requirements

Requirement	Acceptance criterion
FR-01, FR-02	Given valid details, the farmer reaches the dashboard. Given an empty or malformed email or a short password, an error is shown and no account is created.
FR-03, FR-04	Given a valid name, town, size and location, the farm is saved and still shown after a reload.
FR-07, FR-08	Given a saved farm location, the dashboard shows current weather and seven daily entries, or a clear error with retry.
FR-09	Given a forecast with a value at or above a threshold, a warning with action, timing and reason is shown. Given a value just below, no warning is shown.
FR-11, FR-12	Only reviewed and published lessons are listed. Completing a lesson adds an entry to the farm memory.
FR-14, FR-15	A note outside 25 km is refused. A second report of the same note by the same farmer is refused. The second distinct report hides the note.

5. SYSTEM DESIGN SPECIFICATION

5.1 Use Cases and Workflows

Use case diagram

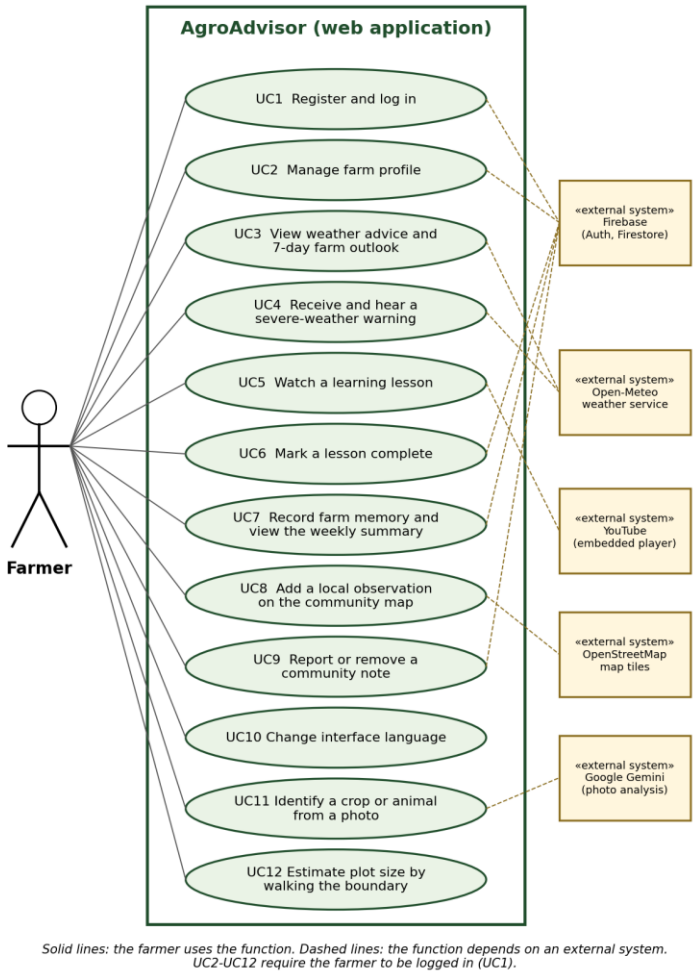


Figure 2: Use case diagram (the farmer is the only actor)

Workflow 1: view weather advice and warnings

The farmer opens the dashboard. The client asks the Worker for the forecast. If the request fails, an error with a retry is shown. If the forecast arrives, the advice engine compares it with the knowledge-base thresholds and shows advice, and a warning if a threshold is reached.

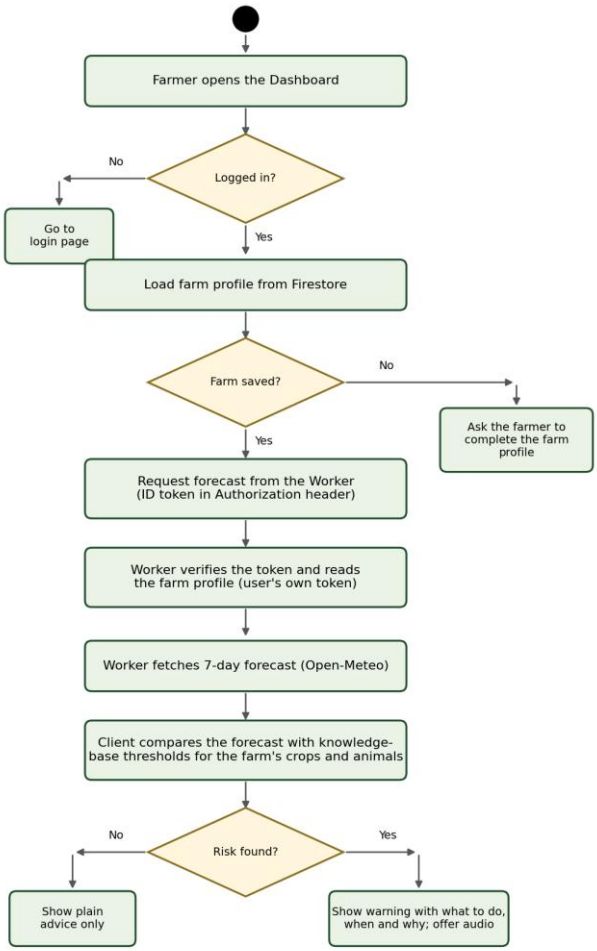


Figure 3: Activity diagram: view weather advice and warnings

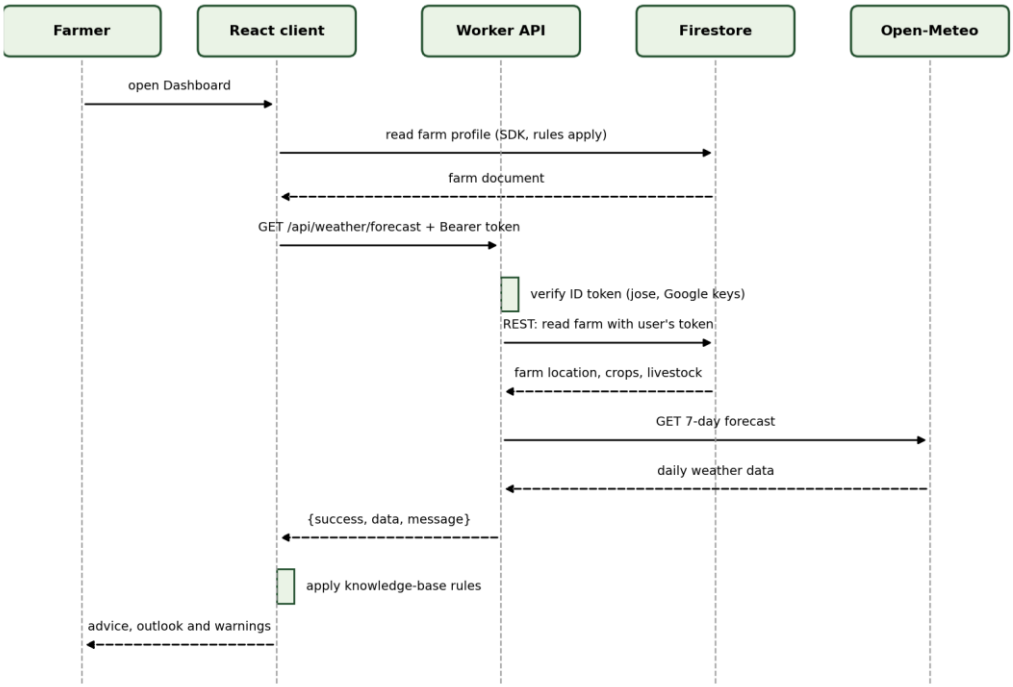


Figure 4: Sequence diagram: load farm weather advice

Workflow 2: report a community note

A farmer reports a note they believe is wrong. The rules refuse a report on the farmer's own note and a second report by the same farmer. The report and the count increase happen in one transaction, so either both are saved or neither is.

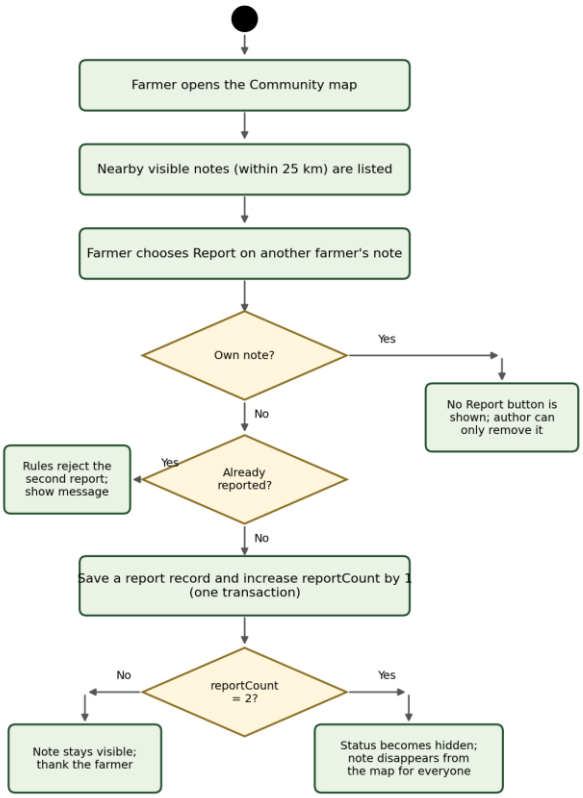


Figure 5: Activity diagram: report a community note

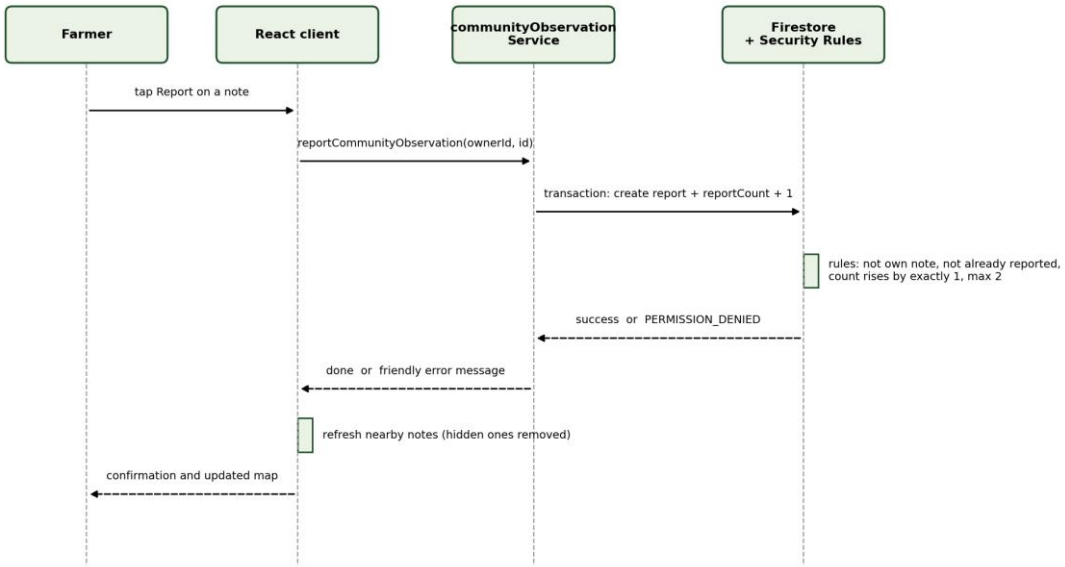


Figure 6: Sequence diagram: report a community note

State diagram

A community note is visible when created and hidden after two reports. It can be deleted by its author at any time.

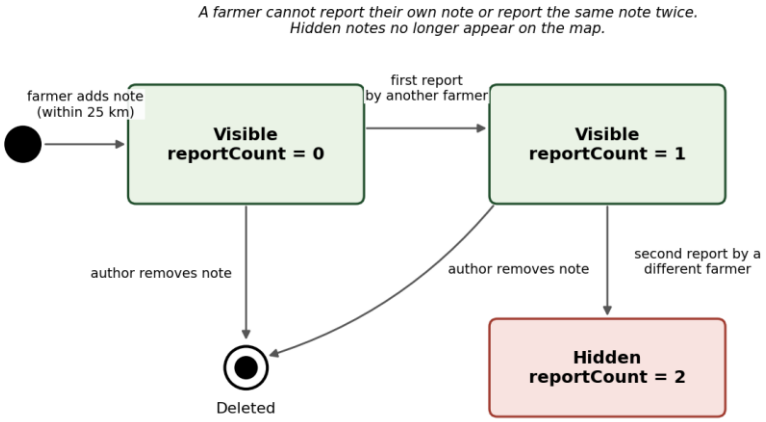


Figure 7: State diagram of a community note

5.2 Architecture and Technology Stack

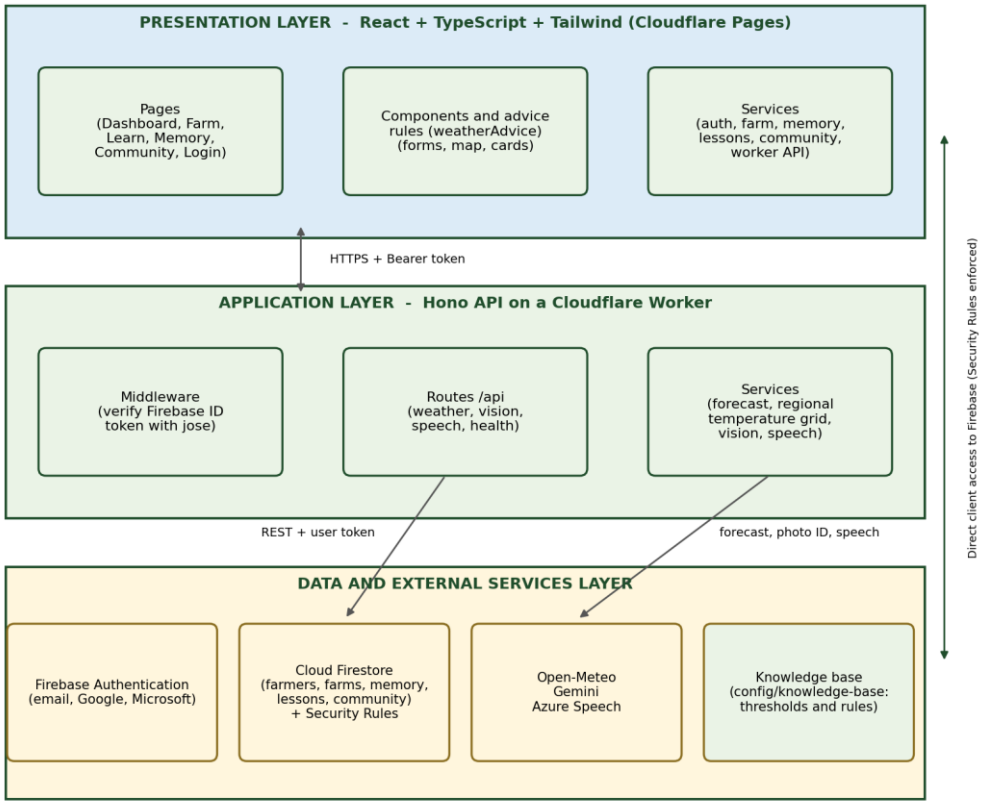


Figure 8: Logical architecture

Component	Responsibility
React client	Screens, input checking, advice rules, and reading and writing the farmer's own Firestore data through service modules.
Worker API (Hono)	Verifies the Firebase ID token, reads the farm with the farmer's token, fetches forecasts, identifies photos and creates speech. All routes are under /api and answer

Component	Responsibility
	with { success, data, message }.
Firebase Authentication	Creates accounts and signs farmers in (email and Google).
Cloud Firestore and Security Rules	Stores data and decides who may read or write each document.
Knowledge base	Holds all agronomic thresholds with a comment explaining each rule and its reason.
External services	Open-Meteo (forecast), Google Gemini (photo identification), Azure Speech (spoken warnings), YouTube (lesson videos), OpenStreetMap (map tiles).

Component	Technology	Responsibility
Client	React 19, Vite 8, Tailwind CSS 4, React Router 8, TypeScript 7, react-leaflet	User interface and advice rules
Worker	Hono 4, jose 6, Wrangler 4, TypeScript (Cloudflare Workers runtime)	Authenticated API
Identity	Firebase Authentication (SDK 12)	Sign-in
Database	Cloud Firestore with Security Rules	Storage and access control
Weather	Open-Meteo forecast API (no key)	Forecast data
AI	Google Gemini (vision)	Photo identification
Speech	Azure Speech F0 (af-ZA, zu-ZA voices); browser voices for other languages	Spoken warnings
Testing	node:test, Firebase emulator, rolldown	Unit and rule tests

5.3 Key Design Decisions

Decision 1: the client reads and writes Firestore directly

Item	Detail
Context	Farmers need their own data fast, and the project must stay on free plans.
Options	(a) Route all data through the Worker; (b) call Firestore directly from the client protected by Security Rules.
Decision	Option (b).
Rationale	Security Rules enforce permissions on the server, so no client code has to be trusted, and no extra API is needed for simple reads and writes.
Consequences	Rules must be tested carefully, which is why six rule tests exist. Complex logic such as the report count had to be expressed as rules.

Decision 2: the Worker calls Firestore with the farmer's own token

Item	Detail
Context	The Worker needs the farm location to fetch a forecast.
Options	(a) Use a service account that can read everything; (b) forward the farmer's ID token to the Firestore REST API.
Decision	Option (b) for user requests. A service-account helper exists only for background work.
Rationale	The same Security Rules still apply, so the Worker can never read another farmer's farm.
Consequences	Each request needs a valid token, and background jobs need a service-account key (not yet configured).

Decision 3: a rule-based advice engine with a knowledge base

Item	Detail
Context	Advice must be explainable and safe for farmers.
Options	(a) Ask a language model for advice; (b) compare forecasts with fixed thresholds kept in a knowledge-base file.
Decision	Option (b). Gemini is used only to identify a photo.
Rationale	Thresholds are transparent, testable and can be changed in one place. Each rule has a comment giving its meaning and reason.
Consequences	Advice only covers the rules and crops that are written down.

Decision 4: Gemini only for photo identification, with manual fallback

Item	Detail
Context	Typing a crop or animal name is slow on a phone.
Options	(a) Typed names only; (b) photo identification with a typed fallback.
Decision	Option (b), with a model fallback list and a confidence rule.
Rationale	A photo is faster, and the confidence rule marks weak results as uncertain or unknown.
Consequences	It needs an API key and may be wrong, so the farmer can always edit or type the name.

5.4 Application Class Design

Figure 9 shows the main services and logic modules of the application, not only the data. Client services are the only code that touches Firestore. The advice engine reads the knowledge base. WorkerApi sends the farmer's token to the Worker. In the Worker, the authentication middleware guards every route, the routes call the worker services, and every route answers through the shared response helper.

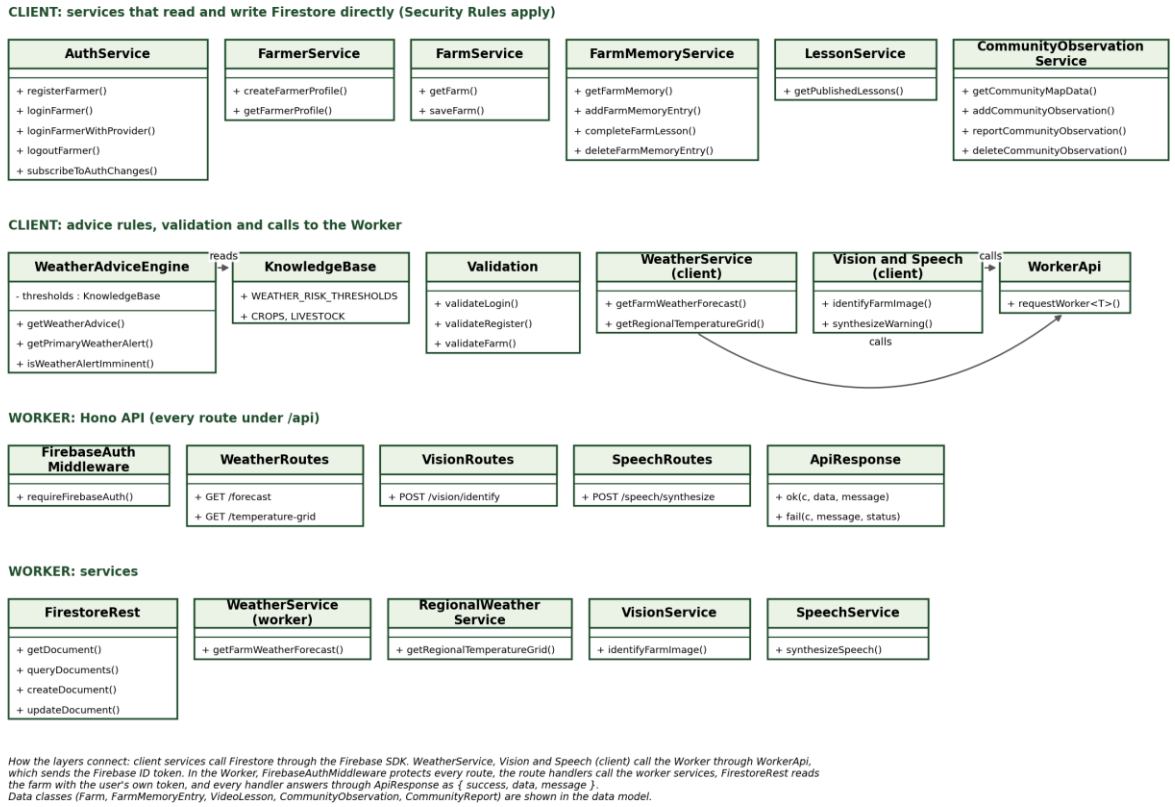


Figure 9: Application class diagram (client and Worker)

The data classes (farmer, farm, memory entry, lesson, community observation and report) are shown in the data model below.

5.5 User Interface and Interaction

Screen	User	Purpose	Action	Requirement
Login (Figure 10)	Farmer	Sign in	Enter email and password or choose Google	FR-01, FR-02
Farm validation error (Figure 11)	Farmer	Show that bad input is refused	Clear the name, enter size 0, press Save	BR-02
Dashboard (Figure 12)	Farmer	Core workflow and result: weather, outlook and warning	Open the dashboard	FR-07 to FR-09
Warning pop-up (Figure 13)	Farmer	Result: what to do, when and why	Open the warning	FR-09
Farm profile (Figure 14)	Farmer	Save the farm	Fill in and save	FR-03, FR-04

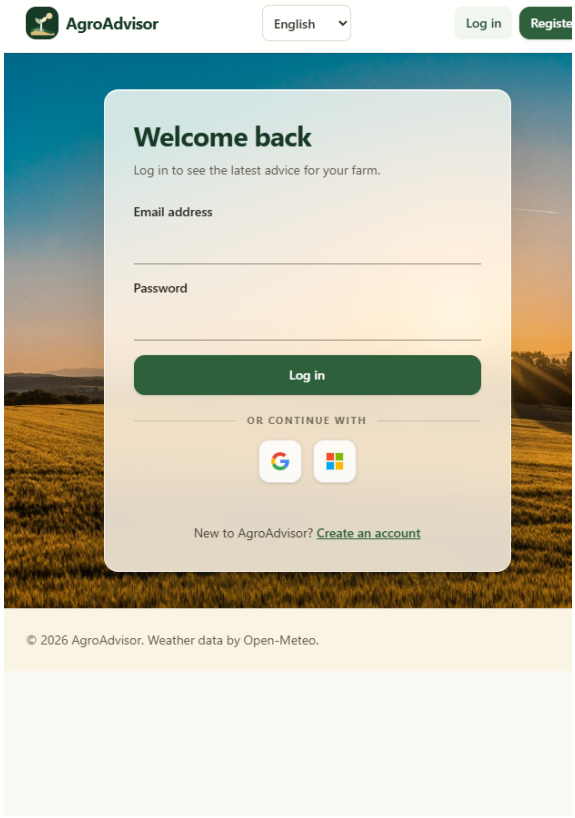


Figure 10: Login screen

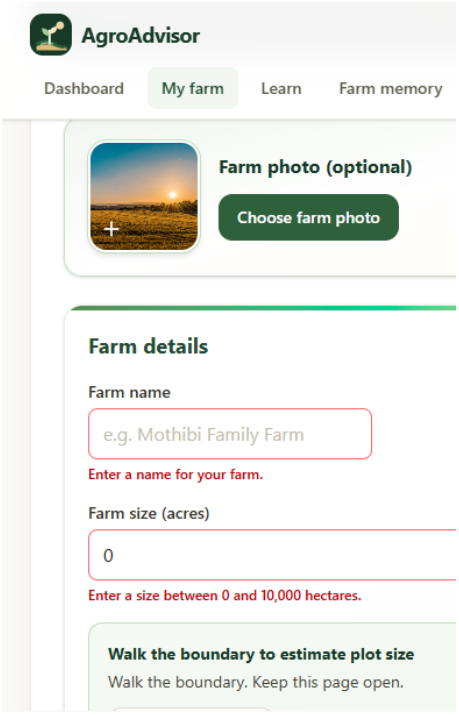


Figure 11: Validation state: the farm form refuses an empty name and a size of 0 (captured during the live test)

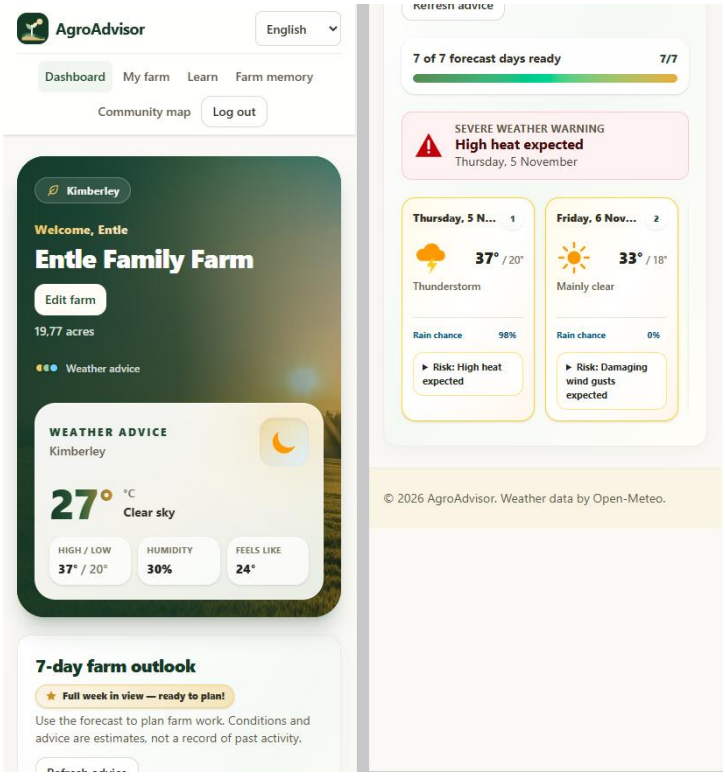


Figure 12: Dashboard: farm, weather, seven-day outlook and warning icon

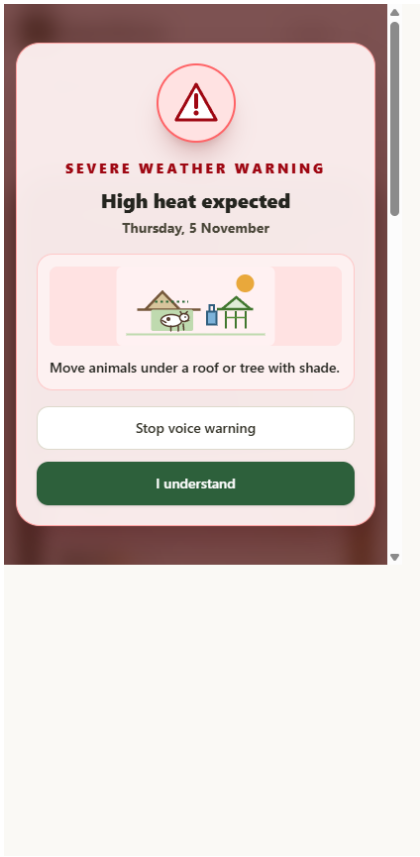


Figure 13: Result state: severe-weather warning pop-up

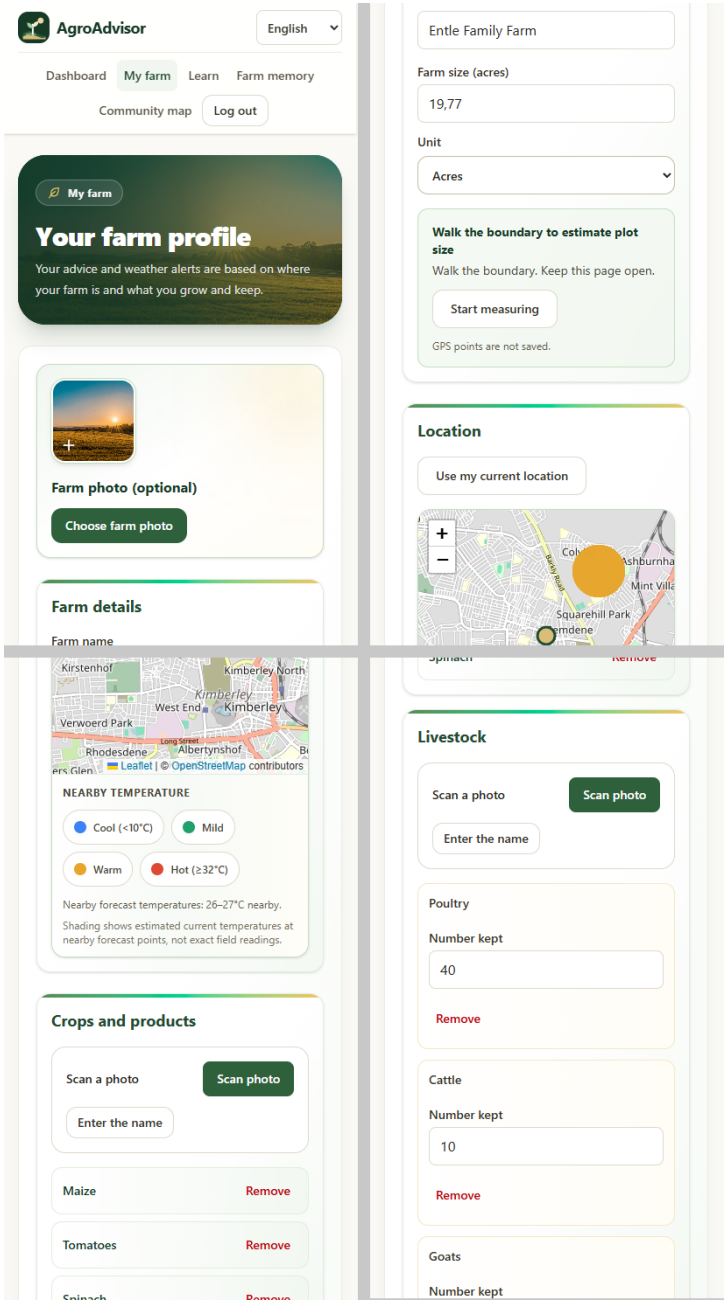


Figure 14: Farm profile

5.6 Data Design

Data is stored in Cloud Firestore. Figure 15 shows the entities and their relationships.

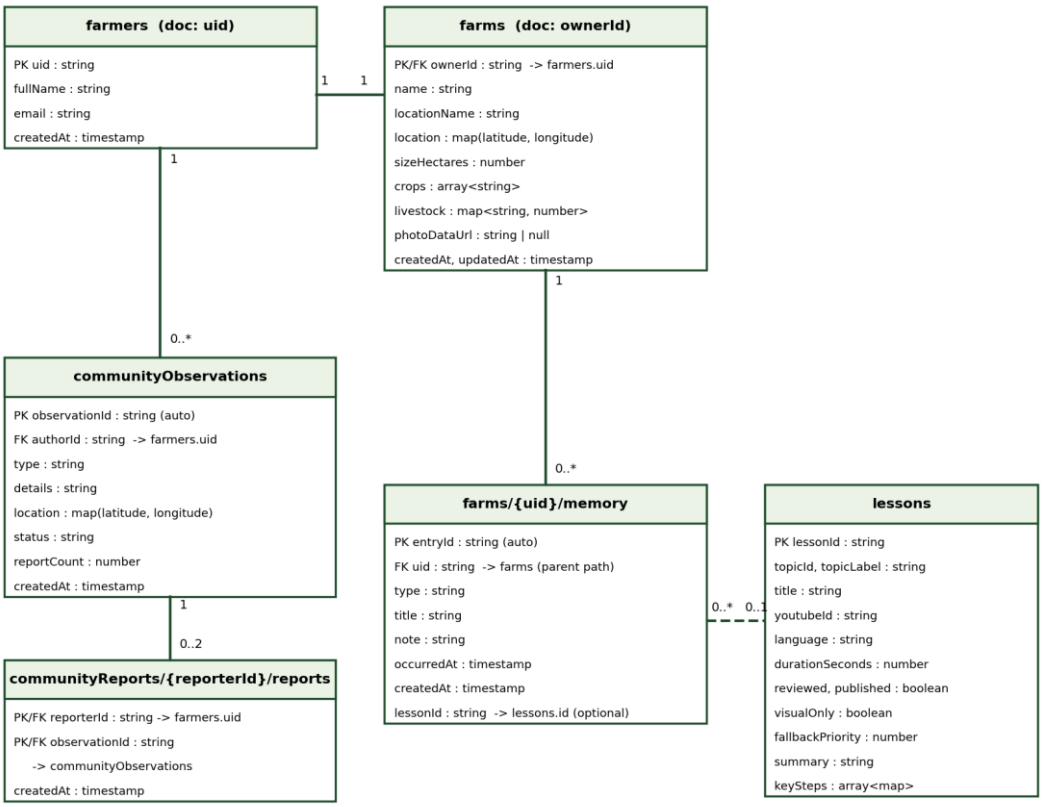


Figure 15: Data model

Collection / path	Key fields	Rules and limits
farmers/{uid}	uid, fullName, email, createdAt	Owner only. Email must match the account.
farms/{ownerId}	name, locationName, location, sizeHectares, crops, livestock, photoDataUrl, updatedAt	Owner only. Size above 0 and at most 10,000 ha. Name 1 to 80 characters.
farms/{uid}/memory/{entryId}	type, title, note, occurredAt, createdAt, lessonId	Owner only. Create or delete, no edit. Type is weather, planting, success or lesson. Date not in the future.
lessons/{lessonId}	topicId, title, youtubeId, language, reviewed, published, keySteps	Signed-in read of reviewed and published only. No writes by farmers.
communityObservations/{id}	authorId, type, details, location, status, reportCount, createdAt	Within 25 km of author's farm. Details at most 240 characters. Count 0 to 2.
communityReports/{reporterId}/reports/{observationId}	createdAt	Reporter only. One per note. Cannot be changed or deleted.

Integrity mechanisms: the farm and the farmer share one ID, so a farmer has one farm; the report action is a Firestore transaction that saves the report and increases the count together; rules check

that the count rises by exactly 1 and never exceeds 2; the report document ID equals the note ID, so a duplicate report is impossible; and server timestamps stop the client from setting creation times.

5.7 Security, Privacy and Access Control

Authentication uses Firebase Authentication with email and password or Google. The Worker verifies the ID token with the jose library against Google's public keys, checking issuer and audience. There is a single role, the farmer.

Resource	Farmer (own data)	Another farmer	Not signed in
Farmer profile and farm	Read, write	No access	No access
Farm memory	Read, create, delete	No access	No access
Lessons (published)	Read	Read	No access
Community notes (visible)	Read, create, delete own	Read, report once	No access
Worker API (/api/weather, vision, speech)	Allowed with a valid token	Not applicable	401 Unauthorized

Area	Risk	Implemented control
Data access	A farmer reads or edits someone else's farm	Firestore Security Rules on every collection; Worker uses the farmer's own token
API	Anonymous use of paid or limited services	Firebase ID token required; missing or invalid token returns 401
Community	Fake or repeated reports, abusive notes	One report per farmer per note, count rules, hide at two reports, no names, 25 km limit, length limit
Input	Oversized or wrong files and values	Photo type and 6 MB limit; size and length limits in rules; client and server validation
Secrets	Keys leaked in source code	Secrets only in .env and worker/.dev.vars (Wrangler secrets); example files hold placeholders
Cross-origin	Other websites calling the Worker	ALLOWED_ORIGIN setting limits browser origins
Privacy	Location exposure	GPS boundary points are not saved; community notes carry no names; farm data is private to the owner

5.8 Deployment and Runtime Design

The assessed system runs on one computer: the Vite development server for the client and the Wrangler development server for the Worker, with the Vite server forwarding /api requests to the Worker. Both talk to managed cloud services over HTTPS.

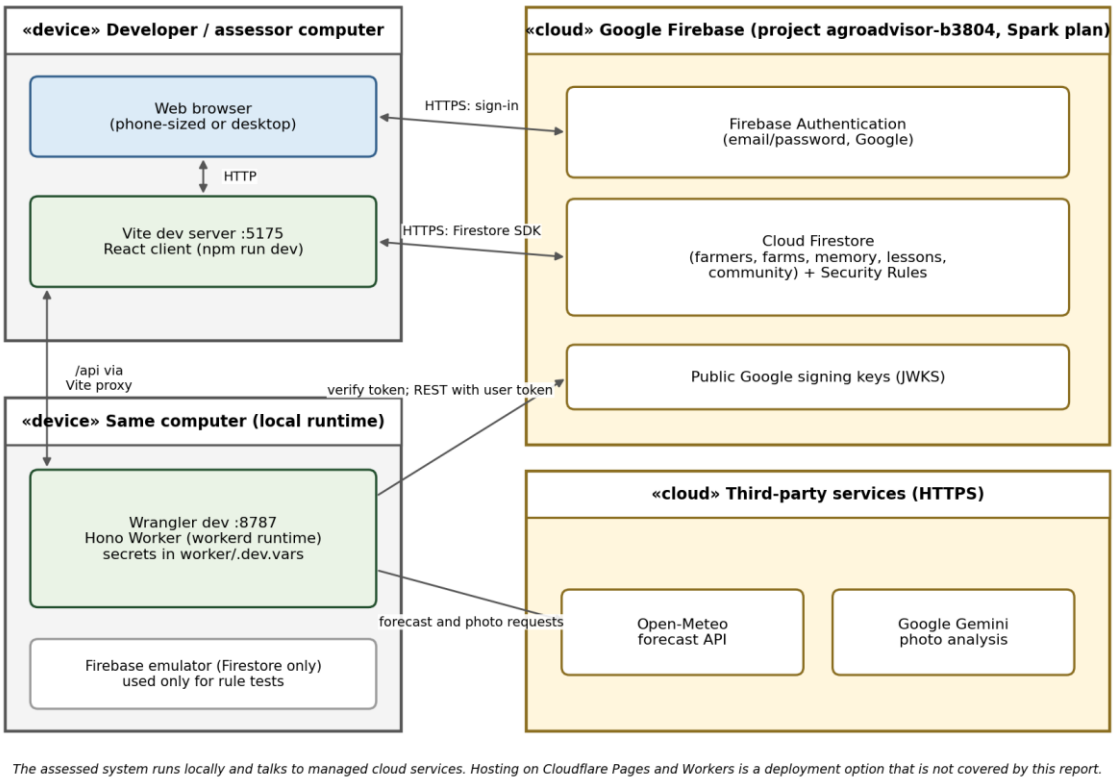


Figure 16: Deployment diagram

6. SYSTEM IMPLEMENTATION

6.1 Codebase and Repository Structure

Folder	Contents
client/	React application: pages, components, hooks, services (all Firestore and Worker calls), types, translations, and tests.
worker/	Hono Worker: routes, middleware, services and types under src/, with wrangler.toml and .dev.vars.example.
config/knowledge-base/	Agronomic thresholds and rules for weather risks, crops and livestock, each with an explanatory comment.
firestore.rules and firestore.indexes.json	Security Rules and query indexes.
docs/	Phase documents, diagrams and the scripts that build them.

6.2 Implementation of Core Capabilities

Capability	Requirement(s)	Main components
Sign-in and registration	FR-01, FR-02	authService, validation module, Login and Register pages
Farm profile	FR-03 to FR-06	farmService, Farm page, map picker, photo scan, plot walker
Weather advice and outlook	FR-07, FR-08	Worker weather route and service, weather advice engine, Dashboard
Severe-weather warning	FR-09	Advice engine, knowledge base, warning pop-up, speech route
Temperature map	FR-10	Regional weather service, temperature-grid route

Capability	Requirement(s)	Main components
Learn and farm memory	FR-11 to FR-13	lessonService, farmMemoryService, Learn and Memory pages
Community map	FR-14, FR-15	communityObservationService, Community page, Firestore rules
Language	FR-16	Translation files and language selector

6.3 APIs and Integrations

Integration	Use	Failure handling
Worker GET /api/weather/forecast	Farm forecast	401 without a valid token; the client shows an error with a retry
Worker GET /api/weather/temperature-grid	Regional temperatures for the map	401 without a token; the map still works without shading
Worker POST /api/vision/identify	Photo identification	400 for a bad type, 413 for oversize, 503 if the service is not configured; the farmer can type the name
Worker POST /api/speech/synthesize	Spoken warning	503 if not configured; the warning stays on screen as text
Open-Meteo	Weather data	A failed request returns an error response and nothing is cached as correct
Firebase Auth and Firestore	Accounts and data	Errors are shown to the farmer in plain language
YouTube and OpenStreetMap	Videos and map tiles	Loaded only when needed; a failure does not affect advice

No integration is mocked. Which of the failure responses above were measured is stated under Verification and Testing.

6.4 AI/ML Component

The only AI component is Google Gemini vision, used to identify a crop or animal from a photo. The Worker tries a list of models in order (gemini-3.1-flash-lite, gemini-3.5-flash-lite, gemini-3.6-flash) and returns a name with a confidence value. A confidence of 0.8 or more is 'identified', 0.5 to 0.79 is 'uncertain' and lower, or a non-farm picture, is 'unknown'. Limitations: the model can be wrong for similar plants or poor photos, it needs a network and an API key, and no accuracy figure has been measured. Fallback: the farmer can always type the name, and every identified item can be edited before saving. The advice and risk engine is not AI; it is rule-based so that every warning can be traced to a threshold.

7. VERIFICATION AND TESTING

7.1 Test Approach

Automated checks: the client production build, strict type checks for the client and the Worker, 15 unit tests for the advice and validation logic, and 6 Firestore Security Rule tests run against the Firebase emulator. Scripted API checks were made against the local Worker. Manual checks were made in a real browser. Items that were not run are labelled 'Not yet executed'. Nothing in this section is estimated.

7.2 Functional and Business-Rule Verification

Test ID	Requirement	Scenario	Type	Expected	Actual	Status
TC-U01	FR-09, BR-03	Temperature 35 °C	Unit	Heat warning	Heat warning raised	Pass
TC-U02	BR-03	Temperature 34.9 °C	Unit	No heat warning	None raised	Pass
TC-U03	BR-03	Temperature 2 °C	Unit	Frost warning	Frost warning raised	Pass
TC-U04	BR-03	Rain 50 mm, and 30 mm at 70%	Unit	Heavy-rain warning	Raised in both cases	Pass
TC-U05	BR-03	Gusts 60 km/h	Unit	Wind warning	Wind warning raised	Pass
TC-U06	BR-03	Weather codes 95, 96, 99	Unit	Storm warning	Raised for each	Pass
TC-U07	FR-08	Missing (null) weather values	Unit	No crash, no false warning	No crash, none raised	Pass
TC-U08	FR-09	Several risks on one day	Unit	Most serious is primary	Correct order	Pass
TC-U09	BR-04	Event today, tomorrow, later	Unit	Pop-up only today or tomorrow	As expected	Pass
TC-U10 to U15	FR-01, BR-02, SC-4	Login, register and farm validation with invalid input	Unit	Error returned	Errors returned	Pass
TC-R01	BR-07	Same farmer reports one note twice	Rules	Second report refused	Refused	Pass
TC-R02	BR-08	Edit report count without a report	Rules	Rejected	Rejected	Pass
TC-R03	BR-08	Report without the count update	Rules	Rejected	Rejected	Pass
TC-R04	BR-08	Second distinct farmer reports	Rules	Note hidden	Note hidden	Pass
TC-R05	BR-07	Author reports own note	Rules	Refused	Refused	Pass
TC-R06	NFR-03	Another farmer deletes a note	Rules	Refused	Refused	Pass
TC-M01	BR-02, SC-4	Farm form: clear the name, size 0, press Save (live browser)	Manual	Both errors shown, nothing saved	'Enter a name for your farm.' and 'Enter a size between 0 and 10,000 hectares.' shown; after reload the saved farm (Entle Family Farm, 8 ha) was intact	Pass
TC-M02	FR-01	Wrong password and Google sign-in error screens	Manual	Friendly error	-	Not yet executed
TC-M03	FR-14	Add note outside 25 km in the browser	Manual	Refused	-	Not yet executed

Thresholds are tested exactly (35 °C raises, 34.9 °C does not). Rule tests TC-R01 to TC-R06 are the six tests in client/tests/firestore.rules.test.mjs. Unit tests are in client/tests/logic.test.ts and run with 'npm run test:unit'.

7.3 Non-Functional and Security Verification

Test ID	Requirement	Check	Result	Status
TC-N01	SC-5	Client production build (npm run build)	Built successfully	Pass
TC-N02	NFR-05	Client strict type check (tsc -b)	No errors	Pass
TC-N03	NFR-05	Worker type check (npm run typecheck)	No errors	Pass
TC-S01	NFR-03	GET /api/health	200, { success: true }, about 196 ms (single local request)	Pass
TC-S02	NFR-03	GET /api/weather/forecast with no token	401	Pass
TC-S03	NFR-03	GET /api/weather/forecast with an invalid token	401	Pass
TC-S04	NFR-03	POST /api/vision/identify, POST /api/speech/synthesize and GET /api/weather/temperature-grid with no token	401 for each	Pass
TC-S05	NFR-05	Request to an unknown route /api/nothing	404	Pass

Test ID	Requirement	Check	Result	Status
TC-S06	NFR-03	Authenticated requests with a valid token, and 400, 413 and 503 responses	-	Not yet executed
TC-P01	NFR-07	Dashboard load time on a slow mobile connection	-	Not yet executed

The 196 ms figure is one local request and is not a performance measurement.

7.4 Defects and Known Issues

Defects found while building and fixed: Google sign-in failed on 127.0.0.1 until that address was added to Firebase's authorised domains. Unresolved issues are listed in Known Limitations and Open Issues (LIM-1 to LIM-8).

8. SYSTEM SETUP AND OPERATION

8.1 Prerequisites and installation

- Node.js and npm. Java 17 (only for the Firestore rule tests).
- A Firebase project with Authentication (email/password and Google) and Cloud Firestore enabled. Add the dev address (localhost and 127.0.0.1) to the authorised domains.
- Install dependencies: run 'npm install' in client/ and in worker/.

8.2 Environment configuration

Copy the example files and fill in your own values. Never commit the real files.

File	Variable	Placeholder / purpose
client/.env	VITE_FIREBASE_API_KEY, VITE_FIREBASE_AUTH_DOMAIN, VITE_FIREBASE_PROJECT_ID, VITE_FIREBASE_STORAGE_BUCKET, VITE_FIREBASE_MESSAGING_SENDER_ID, VITE_FIREBASE_APP_ID, VITE_FIREBASE_VAPID_KEY (push, optional), VITE_WORKER_URL (empty locally)	your-firebase-web-config-value
worker/.dev.vars	FIREBASE_PROJECT_ID	your-project-id
worker/.dev.vars	ALLOWED_ORIGIN	http://localhost:5173
worker/.dev.vars	GEMINI_API_KEY	your-gemini-key (photo identification)
worker/.dev.vars	GOOGLE_SERVICE_ACCOUNT_EMAIL, GOOGLE_SERVICE_ACCOUNT_PRIVATE_KEY	service-account values for background work (not configured)
worker/.dev.vars	AZURE_SPEECH_KEY, AZURE_SPEECH_REGION	your-azure-speech-key and region (spoken warnings)

Check the exact variable names against client/.env.example and worker/.dev.vars.example, which list every required variable.

8.3 Run commands

Purpose	Command (folder)
Start the Worker	npm run dev (worker/), serves on http://127.0.0.1:8787
Start the client	npm run dev (client/); /api is forwarded to the Worker

Purpose	Command (folder)
Build the client	npm run build (client/)
Type-check	npx tsc -b (client/) and npm run typecheck (worker/)
Unit tests	npm run test:unit (client/)
Security rule tests	npm run test:firestore-rules (client/)

8.4 Seeding and demo accounts

The sample data is 15 demo farmers (IDs beginning demo-farmer-) with farms and community notes (IDs beginning demo-obs-). It was written to the database once, and can be deleted by that prefix. Demo logins: [Create or note one test account for the assessor and give the email and password to the supervisor separately. Do not put real passwords in this report.]

9. SOLUTION EVALUATION

Success criterion	Outcome	Evidence
SC-1 Register, sign in, save farm, see advice	Achieved	Live use with a real account; farm saved and still present after reload; validation unit tests pass. Microsoft sign-in is not enabled (Excluded and Deferred Capabilities).
SC-2 Warnings with what, when and why from documented thresholds	Achieved	Unit tests TC-U01 to TC-U09 pass; warning pop-up in Figure 13. Advice only covers rules in the knowledge base.
SC-3 Own-data access and community integrity	Achieved	Six rule tests pass; 401 for unauthenticated API calls. Authenticated API behaviour was not scripted (LIM-3).
SC-4 Invalid input refused	Achieved	Unit tests TC-U10 to TC-U15 and live test TC-M01. The login-error screens were not exercised live (LIM-4).
SC-5 Builds cleanly and can be set up	Partially achieved	Build and both type checks pass. Setup was documented but 5 npm audit findings remain and no public deployment was made (LIM-1, LIM-5).

Overall, the core purpose, interpreting a farm-specific forecast into warnings and advice, has been built and verified at the level shown. The remaining gaps are background push, seasonal advice, measured performance and public deployment.

10. REFERENCES

Department of Agriculture. (2026, September 2). El Niño advisory [Media statement]. https://www.nda.gov.za/images/Newsroom/Media%20Releases/2026/el-nino-advisory-2026-summer-season.pdf

Farmer's Weekly. (2026, August). SA food inflation at a 16-year low. https://www.farmersweekly.co.za/agri-news/south-africa/sa-food-inflation-at-a-16-year-low/

Farmers Mag. (2025, April). Best mobile apps for farmers in South Africa. https://farmersmag.co.za/2025/04/best-mobile-apps-for-farmers-in-south-africa/

Makamane, A. S. (2023). Capacity of extension and advisory services in supporting farmers to adapt to climate change in the Eastern Cape, South Africa [Doctoral thesis, University of the Free State]. http://hdl.handle.net/11660/12426

South African Weather Service. (2026, September 1). South African Weather Service statement on the 2026/27 El Niño [Media statement]. https://www.weathersa.co.za/api/wp/uploads/2026/09/SAWS-ENSO-Statement-002.pdf

Statistics South Africa. (2017). Census of commercial agriculture 2017: Financial and production statistics (Report No. 11-02-01). https://www.statssa.gov.za/publications/Report-11-02-01/Report-11-02-012017.pdf

Statistics South Africa. (2022). Women in agriculture: Six key facts (Census 2022). https://www.statssa.gov.za/?p=19824

USDA Foreign Agricultural Service, International Production Assessment Division. (2016, May). El Niño drought reduces 2015/16 corn prospects in South Africa. https://ipad.fas.usda.gov/highlights/2016/05/SouthAfricaElNino/index.htm

World Meteorological Organization. (2026). El Niño set to become very strong, raising risks of extreme weather in 2027. https://wmo.int/news/media-centre/el-nino-set-become-very-strong-raising-risks-of-extreme-weather-2027

11. APPENDICES

11.1 Appendix A: Source code repository

GitHub repository URL: [Paste the repository URL]

Release tag or commit: [Paste the tag or commit hash that was assessed]

11.2 Appendix B: Deployed system

No public deployment. The assessed system runs locally as described under System Setup and Operation.

11.3 Appendix C: Supporting material

- Phase 1, Phase 2 and Phase 3 documents (problem, requirements and models, GUI and data design).
- Test sources: client/tests/logic.test.ts and client/tests/firestore.rules.test.mjs.
- Knowledge base: config/knowledge-base/weather-risks.ts, crops.ts and livestock.ts.